
Planning & Design Document

for

Zeldle

Version 1.2

Prepared by
Aidan Hafer

*This application is currently in development.
Everything in this document is subject to change.*

Application Planning.....	1
Tech Stack.....	2
Feature Roadmap.....	2
Initial Release.....	2
Future Updates.....	2
Database Schema.....	2
items.....	2
games.....	3
guesses.....	3
Endpoints.....	4
GET /items.....	4
POST /guess.....	4
GET /game.....	5
Data Model.....	6
Loading & Caching Game Progress.....	7
Security & Rule Enforcement.....	7
Item Selection Algorithm.....	8
Game Design.....	8
Feedback.....	8
Name.....	9
Game.....	9
Purpose.....	9
Consumption.....	9
Acquisition.....	10
Range.....	10
Enemy Interaction.....	10
Control Mode.....	10
Item Roster.....	11

Application Planning

Zeldle is a web application inspired by *Wordle* in which users guess an item from *The Legend of Zelda* series. After each guess, players will be provided with gameplay-oriented feedback based on the item's in-game properties, requiring them to possess knowledge of how an item functions rather than deducing the item based on visual descriptions.

Tech Stack

React (TypeScript) + Vite – Interactive frontend and UI. React was selected for its widespread use and support of components. TypeScript was chosen for its static typing and support of interfaces/types.

Spring Boot – REST API and game logic. Selected because of its widespread industry use and my existing familiarity with Java.

PostgreSQL – Persistent storage for items and game data. Selected for its support of custom enumerated types and compatibility with Spring Boot.

Docker – Deployment. Selected to gain experience with containerized application deployment.

Feature Roadmap

This is an estimation and subject to change. Depending on user feedback, game rules may be updated or requested features may be added.

Initial Release

- Daily puzzles.
- Searchable item list.
- Property-based feedback.
- Win/loss detection.
- Ocarina of Time themed UI.

Future Updates

- Player statistics.
- Additional themes based on different Zelda games.
- Possible rework of game rules and/or item properties.

Database Schema

items

Name	Type	Constraints
id	SERIAL	PRIMARY KEY
name	VARCHAR(50)	NOT NULL
game	GAME (ENUM)	NOT NULL
purpose	PURPOSE (ENUM)	NOT NULL
consumption	CONSUMPTION (ENUM)	NOT NULL
acquisition	ACQUISITION (ENUM)	NOT NULL
range	RANGE (ENUM)	NOT NULL
enemy_interaction	ENEMY_INTERACTION (ENUM)	NOT NULL
control_mode	CONTROL_MODE (ENUM)	NOT NULL
CONSTRAINT one_name_per_game UNIQUE (name, game)		

This table stores each guessable item and its properties.

games		
Name	Type	Constraints
date	DATE	PRIMARY KEY
target_item_id	INTEGER	REFERENCES items(id)

This table stores the date and target item ID of each game. Used as the source of truth for target item IDs and in the item selection algorithm to prevent repeat target items.

guesses		
Name	Type	Constraints
id	SERIAL	PRIMARY KEY
player_id	UUID	NOT NULL

game_date	DATE	REFERENCES games(date) ON DELETE CASCADE
guessed_item_id	INTEGER	REFERENCES items(id)
time_of_guess	TIMESTAMPTZ	DEFAULT NOW()
CONSTRAINT no_duplicate_guesses UNIQUE (player_id, game_date, guessed_item_id)		

This table stores user guesses. Used for rule and game state enforcement.
The player ID will be a randomly generated UUID.

Endpoints

GET /items

Returns a list of the IDs and names of all items in the database.

```
[
  {
    "id": 1,
    "name": "Deku Stick"
  }
]
```

POST /guess

Given the ID of an item, evaluates the guess and returns both the Results and ItemProperties of the guessed item. During evaluation, given a player ID, a check of the “guesses” table in the database will be done to enforce guess limits, prevent duplicate item guesses, and update the remaining guesses.

Request:

```
{
  "itemId": 1
}
```

Response:

```
{
  "gameState": {
    "guessesRemaining": 5,
```

```

    "gameStatus": "IN_PROGRESS"
  },
  "results": {
    "name": "INCORRECT",
    "game": "INCORRECT",
    "game_release_date": "TARGET_HIGHER",
    "purpose": "CORRECT",
    "consumption": "CORRECT",
    "acquisition": "INCORRECT",
    "range": "CORRECT",
    "enemyInteraction": "CORRECT",
    "control_mode": "INCORRECT"
  },
  "properties": {
    "name": "Deku Stick",
    "game": "Ocarina of Time",
    "purpose": "INTERACTION",
    "consumption": "MATERIAL",
    "acquisition": "OVERWORLD",
    "range": "CLOSE",
    "enemyInteraction": "DAMAGE",
    "controlMode": "IMMEDIATE"
  }
}

```

GET /game

Checks the database for a game with today's date (EST). Given the player ID, it returns the date of the current game, as well as the current game state, which includes GuessedIDs, GuessHistory (ordered using time_of_guess), and guesses remaining.

This endpoint will create a new game IF AND ONLY IF the scheduled game creation were to fail. As it is a failsafe and not a core feature, I feel justified keeping this endpoint as a GET.

```

{
  "date": "2026-08-03",
  "gameState": {
    "guessesRemaining": 5,
    "gameStatus": "IN_PROGRESS"
  },
  "guessedIds": [
    1
  ],
  "guessHistory": [
    {

```

```

    "results": {
      "name": "INCORRECT",
      "game": "CORRECT",
      "gameReleaseDate": "EQUAL",
      "purpose": "INCORRECT",
      "consumption": "CORRECT",
      "acquisition": "INCORRECT",
      "range": "CORRECT",
      "enemyInteraction": "CORRECT",
      "controlMode": "INCORRECT"
    },
    "properties": {
      "name": "Deku Stick",
      "game": "Ocarina of Time",
      "purpose": "INTERACTION",
      "consumption": "MATERIAL",
      "acquisition": "OVERWORLD",
      "range": "CLOSE",
      "enemyInteraction": "DAMAGE",
      "controlMode": "IMMEDIATE"
    }
  }
]
}

```

Data Model

- **Result** – The result of a comparison between a property of one item and the same property of another.
- **Results** – The results of all properties from a specific item after a guess. Used for color coordination in the feedback grid.
- **ItemProperties** – The properties of a specific item. Used for filling in the properties of a guess in the feedback grid. Also received from the /guess and /game endpoints.
- **Feedback** – Contains the Results and ItemProperties of a specific guess.
- **GuessesRemaining** – The number of guesses remaining in the current game.
- **GameStatus** – The current status of the game, whether it be won, lost, or in progress. Used for win/loss detection.

- **GameState** – Contains the GuessesRemaining and GameStatus of a specific game.
- **GuessedIDs** – An array of numbers containing the IDs of every item guessed in the session. Used for disabling items in the item list after a guess.
- **GuessHistory** – An array of Feedback containing the Results and ItemProperties of each guess. Used for populating the feedback grid.
- **Item** – Contains the ID and name of a specific item. A list of this data is received from the /items endpoint.
- **Guess** – Contains the Feedback of a specific guess and the GameState after that guess was submitted. Received from the /guess endpoint.
- **Game** – Contains the date, GameState, GuessedIDs and GuessHistory of a specific game. Received from the /game endpoint.

Loading & Caching Game Progress

Game progress, more specifically a Game object, is returned from the backend upon page load. When the /game endpoint is called credentials are sent from the frontend, and these credentials are then filtered by the backend, either creating a new or retrieving an existing cookie containing the player ID. The “guesses” table will be searched for entries containing the current game and the player ID, and these entries will be used to get the GameState, GuessedIDs and GuessHistory.

This data will also be cached in localStorage under the key “zeldle_gameCache” and will be updated after each guess. This is done to instantly load game progress before the data can be retrieved from the backend. The result is the removal of the jarring pop-in that otherwise occurs upon page load. In a similar vein, the data used to populate the item list will be cached in localStorage under the key “zeldle_itemCache” and will be updated if/when the item list is updated.

While the cookie approach may seem odd, this was an intentional choice to avoid OAuth or account creation. I want this game to be something players can quickly pull up and enjoy. There’s no need to bloat this otherwise simple application just for the sake of it.

Security & Rule Enforcement

In order to prevent cheating using inspect element, no information about the current game’s target item will be shared to the frontend under any circumstances. The only exception is when the target item is correctly guessed, to populate the feedback grid.

The number of guesses per game is enforced by the backend. When the player guesses an item, the backend checks the “guesses” table in the database. It will look for all entries in the table with the current date and the player ID. If **six** are found, the backend will not evaluate any more guesses.

Preventing the player from guessing the same item multiple times in one game is also enforced by the backend. This is done via a constraint in the “guesses” table. If the player ID, game date, and guessed item ID match an existing entry in the table, a new entry will not be made and the guess will not be processed.

Item Selection Algorithm

In order to prevent target items from repeating across multiple days, a simple algorithm will be run when selecting an item.

Upon game creation, the “items” table will be queried to select a random item, excluding any item IDs present in the “games” table. The item that is randomly selected will then be added to the “games” table. Entries in that table are deleted after 14 days, which will bring that item back into circulation. The result is that an item can be selected AT MOST once every 14 days.

Game Design

In *Zeldle*, players will have **six** attempts to guess a randomly selected item. Players will guess by choosing from a set list of items, and use the feedback provided from previous guesses to deduce the target item.

Feedback

Feedback is provided to players after each guess, and the results of this feedback is designed to narrow down their guesses.

Items have eight unique properties in which they are defined, and as such feedback is divided into eight columns: **Name, Game, Purpose, Consumption, Acquisition, Range, Enemy Interaction**, and **Control Mode**.

After the player makes a guess, the topmost empty row on the feedback grid will be color coded to denote a specific result for each property. The color codes are as follows:

- **Green** denotes a match – The property of the guessed item matches that of the target item.
- **Red** denotes a mismatch – The property of the guessed item does not match that of the target item.

Below are explanations of each of the eight properties and how they define each item:

Name

Refers to the name of an item.

Game

Refers to the game in which an item is found.

If the game of the target item was released after that of the player's guessed item, an up arrow (↑) will be displayed in the column. Likewise, if the game of the target item was released before that of the player's guessed item, a down arrow (↓) will be displayed.

Purpose

Refers to the primary use case of an item. An item can have one of three purposes:

Combat – Refers to items that primarily aid in combat, offensively or defensively.

Interaction – Refers to items that primarily interact with objects in the world or the world itself.

Traversal – Refers to items that primarily open up new traversal options for the player.

Consumption

Refers to the resource an item consumes upon use. An item can consume one of three resources:

Material – Refers to items that consume a finite, physical resource upon use.

Magic – Refers to items that consume magic power upon use.

None – Refers to items that consume nothing upon use.

Acquisition

Refers to the method in which an item is initially acquired. An item can be initially acquired in one of three methods:

Overworld – Refers to items acquired via overworld exploration. This includes non-dungeon interiors such as fairy fountains and grottos.

Dungeon – Refers to items acquired via dungeon crawling.

Purchase – Refers to items acquired via purchase in or outside of a shop.

Range

Refers to the effective range of an item. An item can have one of three ranges:

Self – Refers to items that affect the player rather than any external target.

Close – Refers to items that affect targets within a trivial distance from the player.

Ranged – Refers to items that affect targets at a significant distance from the player.

Enemy Interaction

Refers to the effect an item has on enemies. An item can interact with enemies in one of three ways:

Damage – Refers to items that inflict damage upon enemies.

Disrupt – Refers to items that disrupt enemies in some way, such as stunning, freezing, or even interrupting a pattern.

None – Refers to items that do not interact with enemies at all.

Control Mode

Refers to how an item is operated. An item can be operated in one of three ways:

Immediate – Refers to items that produce an effect immediately upon use.

Targeted – Refers to items that produce a targeted effect the player must aim.

Persistent – Refers to items that produce an effect immediately after use, and continue their effect for a prolonged period.

Item Roster

Items will undergo a series of checks before being included as part of the item roster.

Items will be chosen only from the following games:

- *Ocarina of Time*
- *Majora's Mask*
- *The Wind Waker*
- *Twilight Princess*
- *Skyward Sword*

This is because items from earlier 2D entries and later 3D entries differ dramatically from “classic 3D” Zelda items, making them incompatible with any unified feedback system.

Even in the limited range of games there are many items that are either extreme outliers or identical to one another in a number of aspects. Including every item from the range of games would require an excessive number of item properties, extremely specific cases in the existing properties, or both.

A spreadsheet of all included items, their properties, explanations for their assigned properties, and explanations for all excluded items will be made publicly available during the initial v1.0 release. This will allow for community members to give their feedback and allow for any needed adjustments.